

Report
Of
Food Genie – AI Food Ordering App

Submitted To:
Ms. Surabhi Purwar

School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of
B.Tech CSE



IILM University
Greater Noida, Uttar Pradesh

By
DEVANSHU VERMA
Roll Number: CS-23411461
Batch: 2023–2027
Internship Organization: Webstack Academy (WSA)
Role: MERN Stack Developer
Internship Duration: 3 June 2026 – 3 July 2026
2026–27

CANDIDATE'S DECLARATION

I, Devanshu Verma do hereby declare that the internship report titled “Food Genie-AI Food Ordering App” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature

Student Name- Devanshu Verma

Roll No.- CS-23411461

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I express my gratitude to my parents for their blessings.

Signature

Student Name- Devanshu Verma

Roll No.- CS-23411461

TABLE OF CONTENTS

1. Candidate's Declaration
2. Acknowledgement
3. Internship Completion Certificate
4. Project Description
 - 4.1 Introduction
 - 4.2 Organization Profile
 - 4.3 Problem Statement
 - 4.4 Project Objectives
 - 4.5 Scope of the Project
 - 4.6 Technologies and Tools Used
 - 4.7 System Architecture
 - 4.8 Methodology
 - 4.9 Expected Outcomes
 - 4.10 Certificates of Completion and Communication Proof
5. Bibliography / References

4. PROJECT DESCRIPTION

4.1 Introduction

Food Genie is a full-stack food ordering platform developed using the MERN stack. The application brings restaurant discovery, menu browsing, food-item selection, cart management, authentication, checkout and order processing into a connected workflow. The project was developed during the internship at Webstack Academy (WSA) in the role of MERN Stack Developer.

The frontend is implemented using React 18 with Vite, React Router and Redux Toolkit. Axios is used for communication with the backend REST APIs. The backend is built using Node.js and Express.js and follows a modular structure based on routes, controllers, middleware, models and utilities. MongoDB with Mongoose is used for persistent storage.

The application also integrates Stripe for checkout and payment flow, Cloudinary for media storage, and Nodemailer for email-related functionality. JWT-based authentication and bcrypt password hashing are used as part of the application's security mechanisms.

4.2 Organization Profile

The internship was undertaken at Webstack Academy (WSA), where the assigned role was MERN Stack Developer. The internship provided practical exposure to full-stack web application development through the implementation of the Food Genie project. The work involved frontend development, backend API development, database modeling, authentication, payment integration and integration with external cloud services.

The information in this section is limited to the internship and project details available for this report. Additional official organizational information, if required by the university, should be inserted from the organization's official profile or internship documentation.

4.3 Problem Statement

Traditional food-ordering project implementations may separate browsing, cart management, authentication and payment into disconnected modules. A complete application requires consistent data flow across the client and server, secure APIs, persistent data storage and reliable integration with third-party services.

Food Genie addresses this development problem by connecting the major food-ordering functions into a single end-to-end workflow. The system provides restaurant and menu browsing, food-item selection, cart management, checkout, online payment and order persistence through a coordinated MERN-based architecture.

4.4 Project Objectives

Build a complete MERN-based food-ordering workflow.
Provide secure user authentication and protected APIs.
Manage restaurants, menus, food items, carts and orders.
Integrate online payment through Stripe.
Support cloud-based media storage through Cloudinary.
Provide email functionality using Nodemailer.
Maintain a modular backend structure that can be extended with future recommendation and AI features.

4.5 Scope of the Project

The implemented scope of Food Genie includes the following capabilities:

Restaurant discovery and menu browsing.

Food-item viewing and selection.

Cart creation and quantity/total management.

User registration and login.

JWT-based protected API access.

Order placement and order history.

Stripe-hosted checkout and payment workflow.

MongoDB-based persistence of users, restaurants, menus, carts and orders.

Cloudinary-based media handling.

Email utility and password-reset communication capability.

AI/ML-based personalized recommendations, demand forecasting and conversational ordering are considered future extensions in the project material. They should not be presented as implemented functionality unless the corresponding model or API is actually present in the codebase.

4.6 Technologies and Tools Used

Category	Technology / Tool	Purpose
Frontend	React 18, Vite	User interface and frontend build
Frontend	React Router	Client-side navigation
Frontend	Redux Toolkit	Centralized application state and async actions
Frontend	Axios	HTTP/API communication
Frontend	Bootstrap / styled-components	UI styling
Backend	Node.js, Express.js	Server runtime and REST API
Backend	Middleware / Controllers	Authentication, authorization, business logic and error handling

Database	MongoDB	Persistent application data
Database	Mongoose	Schemas, models, validation and related-data population
Security	JWT	Token-based authentication
Security	bcrypt	Password hashing
Payment	Stripe	Checkout and payment processing
Media	Cloudinary	Cloud media/image storage
Email	Nodemailer	Email and password-reset communication

The technology stack and integrations above are based on the project presentation, which identifies React 18/Vite, React Router, Redux Toolkit, Axios, Node.js, Express, Mongoose, MongoDB, Stripe, Cloudinary, Nodemailer, JWT and bcrypt as project technologies and integrations.

4.7 System Architecture

Food Genie follows a client–API–business logic–database architecture with external service integrations. The React user interface and Redux Toolkit state management communicate with the backend through Axios and REST APIs. Express.js routes and middleware receive requests and pass them to controllers containing business logic. Mongoose models provide the interface to MongoDB. External services are accessed through the backend boundary.

The major architectural flow is:

React UI / Redux Toolkit → Axios / REST API → Express.js Routes & Middleware → Controllers → Mongoose Models → MongoDB

External integrations: Stripe for payment, Cloudinary for media storage and Nodemailer for email functionality.

The backend follows the design pattern: HTTP request → route → middleware → controller → model/service → response.

4.8 Methodology

The project was organized as a modular full-stack application. The frontend was developed as a component-driven React application with client-side routing and centralized application state. Redux Toolkit was used for state and asynchronous actions, while Axios handled communication with REST endpoints.

The backend was structured around routes, controllers, middleware, models and utility/configuration modules. Routes mapped HTTP endpoints to controllers,

middleware handled authentication, authorization and centralized error handling, and Mongoose models represented the core application entities.

The principal database entities are User, Restaurant, Menu, FoodItem, Cart and Order. MongoDB provides the flexible document model, while Mongoose supplies schemas, validation and population for related data.

The implemented ordering workflow consists of six main stages: browsing restaurants and menus, selecting food items, reviewing the cart, creating a Stripe checkout session, completing the Stripe-hosted payment, and persisting the order while clearing the cart. For security, the application uses bcrypt for password hashing, JWT for token generation, protected routes with middleware verification, environment variables for secrets, CORS configuration and centralized error handling. Stripe acts as the payment boundary for payment collection, while the application receives the resulting payment/session information for order processing.

4.9 Expected Outcomes

A functional full-stack food-ordering workflow connecting frontend, backend and database layers.

Improved understanding of React component development and centralized state management.

Practical experience in developing REST APIs using Node.js and Express.js.

Experience in MongoDB data modeling using Mongoose.

Implementation experience with JWT authentication and bcrypt password hashing.

Practical exposure to Stripe checkout and payment integration.

Experience with Cloudinary media handling and Nodemailer-based email functionality.

A modular foundation that can later be extended with automated testing, monitoring, analytics, deployment automation and AI-based recommendations.

The project material also identifies engineering improvements before production deployment, including automated unit/integration testing, API monitoring and structured logging, rate limiting, stronger validation, production CORS and deployment-secret configuration, API performance benchmarking, CI/CD and deployment health checks.

4.10 Certificates of Completion and Other Communication Proof

As required by the university report format, the internship completion certificate issued by Webstack Academy (WSA) and relevant communication/email proof should be attached in this section. The documents should be inserted after this page in their original form.

Suggested attachments:

Internship Completion Certificate – Webstack Academy (WSA).

Internship offer/confirmation communication, if available.

Relevant internship communication/email proof, if required.

Project screenshots or other evidence requested by the university, if applicable.

4.11 Project Implementation and Functional Modules

The implementation of Food Genie was organized into independent but connected functional modules. This modular approach supports maintainability and allows individual parts of the application to evolve without requiring major changes to unrelated components.

User Authentication Module

The authentication module provides registration and login functionality. User credentials are processed with bcrypt password hashing, while JWT is used for token generation. Protected API middleware verifies authentication before allowing access to restricted operations. The authenticated user is made available through the request context.

Restaurant and Menu Module

The restaurant and menu functionality supports restaurant discovery and menu browsing. MongoDB collections and Mongoose models represent restaurant and menu information, while the React frontend provides user-facing pages for navigating this content.

Food Item and Cart Module

Food items contain item details and pricing information. Selected items are maintained through the cart workflow, where users can review quantities and totals before proceeding to checkout.

Order and Payment Module

The order workflow connects the cart with Stripe checkout. The backend creates a checkout session, Stripe hosts the payment flow, and the resulting payment/session information is used during order processing. After successful checkout, the order controller validates the session and persists the order in MongoDB while the cart is cleared.

Media and Email Services

Cloudinary is used for cloud-based media handling, including user/avatar image uploads. Nodemailer provides an email utility and supports password-reset communication and transactional messaging capability.

Frontend Implementation

React components provide the user interface for Home, Restaurant, Menu, Food Item, Cart and authentication views.

React Router provides client-side navigation between application pages.

Redux Toolkit provides centralized application state and asynchronous actions.

Axios communicates with the backend REST APIs using HTTP requests and credentials.

4.12 Testing, Engineering Quality and Limitations

The project was assessed from an engineering perspective with emphasis on modularity, security mechanisms, data flow and third-party service integration. The project presentation identifies several practices that were implemented in the current application.

Implemented Engineering Practices

Modular backend structure organized around routes, controllers, models and middleware.

Protected API routes with authentication and authorization middleware.

JWT-based authentication and bcrypt password hashing.

Centralized error handling.

Environment-based configuration for secrets.

Separate frontend and backend layers.

Integration with external payment and cloud services.

Production Readiness Improvements

The project presentation identifies several improvements that should be addressed before production deployment. These are documented as future engineering work rather than being presented as completed features.

Automated unit and integration tests.

API monitoring and structured logging.

Rate limiting and stronger validation.

Production CORS configuration and deployment-secret management.

Benchmarking API response times before publishing performance metrics.

CI/CD and deployment health checks.

Current AI/ML Boundary

The project material intentionally separates AI/ML claims from the implemented MERN core. Personalized food recommendations, demand forecasting, restaurant insights and conversational ordering are described as future extensions unless the corresponding model or API is implemented in the codebase.

Potential AI/ML Extension

A future recommendation system could use order history and features such as cuisine preference, purchase frequency, price and recency. Content-based or collaborative filtering could then rank menu items for personalized suggestions. This would extend the existing ordering platform rather than changing its core MERN architecture.

5. BIBLIOGRAPHY / REFERENCES

Food Genie project presentation prepared for the project evaluation, containing the project architecture, technology stack, database design, security mechanisms, ordering workflow, integrations, limitations, future scope and conclusion.

React documentation – concepts and APIs used for component-based frontend development.

Node.js documentation – server-side JavaScript runtime concepts.

Express.js documentation – routing and middleware concepts for REST API development.

MongoDB documentation – document database concepts and data modeling.

Mongoose documentation – schemas, models, validation and population.

Stripe documentation – checkout and payment integration concepts.

Cloudinary documentation – cloud media storage and delivery concepts.

Nodemailer documentation – Node.js email functionality.

JSON Web Token (JWT) documentation/specification – token-based authentication concepts.